



PROIECT DE PRACTICĂ

PERSONAL ACADEMIC AI ASSISTANT

Documentație Tehnică de Arhitectură, Implementare și Integrare Sistemică

Student:	Ciobănoiu Rareș-Alexandru
Program de studii:	Tehnologia Informației
Grupa / An de studiu:	Grupa 4LF341 / Anul II
Cadru Didactic Îndrumător:	Șef Lucr. Dr. Ing. Dragoș Bratu
Tutore de Practică (Companie):	[Nume Tutore / Denumire Companie Parteneră]
An Universitar:	2025-2026

Brașov, 2026

CUPRINSUL DOCUMENTAȚIEI TEHNICE

Structura modulară completă a documentației tehnice de proiect (31 capitole tematice):

- 1. Overview & Prezentare Generală
- 2. Domeniul Proiectului (Project Scope & Etape M0-M9)
- 3. Stadiul Real al Implementării (Current Implementation Status)
- 4. Arhitectura Tehnică a Sistemului
- 5. Structura Repository-ului și a Modulelor
- 6. Integrarea Telegram Bot (Webhook & Securitate)
- 7. AI Orchestrator (Rutare și Detecție Intenții)
- 8. Abstracția și Providerii LLM (OpenAI Cloud & Ollama Local)
- 9. Integrarea E-mail (IMAP/SMTP, Clasificare, Human-in-the-Loop)
- 10. Integrarea Instituțională UNITBV & Securitatea Microsoft 365
- 11. Integrarea Google Calendar (API v3 & Parsare Temporală)
- 12. News Agent (Agregator RSS, Scoring și Deduplicare SHA-256)
- 13. Practice Agent (Consultanță Practică Academică)
- 14. Baza de Cunoștințe (Knowledge Base Multi-Anual)
- 15. RAG Pipeline (Chunking, Embeddings, Qdrant Vector Store)
- 16. Arhitectura de Memorie (Conversație, Utilizator, Istoric Practică)
- 17. Managementul Sarcinilor (Action Items)
- 18. Baza de Date și Persistența (PostgreSQL, Modele ORM, Alembic)
- 19. Configurare Centralizată și Variabile de Mediu
- 20. Docker, Containere și Infrastructura de Rulare
- 21. Politici și Mecanisme de Securitate
- 22. Logging, Observabilitate și Tratarea Erorilor
- 23. Suita de Testare Automatizată (91 Teste)
- 24. Scenarii și Fluxuri End-to-End (E2E)
- 25. Ghid de Instalare Pas-cu-Pas
- 26. Ghid de Utilizare și Exemple de Interacțiune
- 27. Ghid de Depanare (Troubleshooting Tehnic)
- 28. Matricea Cerințe vs. Implementare Reală
- 29. Limitări Cunoscute ale Sistemului
- 30. Direcții de Dezvoltare Ulterioară (Remaining Work)
- 31. Concluzii Generale

1. Overview & Prezentare Generală

Sistemul Personal Academic AI Assistant reprezintă o platformă backend modernă, decuplată și modulară, dezvoltată în limbajul Python 3.14 pe baza framework-ului asincron FastAPI. Asistentul este proiectat pentru a deservi ca hub central conversațional pentru studenții și cadrele didactice din cadrul Universității Transilvania din Brașov (UNITBV), automatizând și eficientizând activitățile academice, gestionarea stagiilor de practică studențească, monitorizarea agendei de cursuri/laboratoare și fluxurile de corespondență electronică.

Sistemul elimină bariera interfețelor multiple prin unificarea serviciilor într-un canal accesibil direct de pe dispozitive mobile: aplicația Telegram. În spatele acestui canal, o arhitectură stratificată integrează un motor avansat RAG (Retrieval-Augmented Generation) peste ghidurile oficiale de practică, sincronizare automată de e-mailuri prin IMAP/SMTP, integrare cu Google Calendar API v3 și un agregator inteligent de știri tehnologice.

 **PRINCIPIU FUNDAMENTAL:** Principiul director al sistemului este securitatea și siguranța operațională: nicio acțiune cu impact extern (cum ar fi expedierea unui e-mail de răspuns) nu este realizată automat fără confirmarea explicită a studentului printr-un flux strict de Human-in-the-Loop.

2. Domeniul Proiectului (Project Scope & Etape M0-M9)

Dezvoltarea proiectului s-a realizat incremental, respectând un set clar de jaloane (milestones) tehnice:

- **M0 - Core Infrastructure:** Inițializarea structurii, configurare dinamică cu pydantic-settings, containere Docker.
- **M1 - Telegram Gateway:** Gateway Telegram Webhook cu verificare HMAC a secret token-ului, allow-list și limitare de rată.
- **M2 - Email Integration:** Conexiune IMAP/SMTP duală (personal/UNITBV), căutare de e-mailuri, clasificare prin LLM.
- **M3 - Calendar & News:** Parsare date temporale în limbaj natural pentru Google Calendar și monitorizare fluxuri RSS cu deduplicare SHA-256.
- **M4 - Practice RAG:** Baza de cunoștințe de practică, indexare vectorială în Qdrant, mecanism RAG cu atribuire de surse.
- **M5 - Sinergie Email ↔ RAG:** Generare automată de drafturi de răspuns din ghidul oficial și flux Human-in-the-Loop cu persistență.
- **M6 - Suport Multi-Anual:** Filtrare după anul universitar (ex: 2024-2025, 2025-2026, 2026-2027) și regăsire răspunsuri istorice similare.
- **M7 - Hardening & On-Premise:** Integrare runtime hibrid (OpenAI Cloud + Ollama Local on-premise), backup și restore criptografic (SHA-256).

3. Stadiul Real al Implementării (Current Implementation Status)

Tabelul de mai jos sintetizează cu precizie starea fiecărui modul software identificat în codul repository-ului, diferențiind componentele validate de cele care necesită configurare live externă sau integrare în fluxul principal:

Componentă	Nivel Implementare în Cod	Acoperire Teste	Status Validare
FastAPI Core & Health Probes	Completă (main.py, /health/*)	Teste unitare & integrare	Validat local & Docker
Telegram Webhook Gateway	Completă (telegram.py, HMAC, rate-limit)	Testat cu TestClient & mock	Implementat, nevalidat live
AI Orchestrator	Completă (rutare deterministă pe regex)	Testat unitar cu mock providers	Validat local
OpenAI Provider	Completă (chat completions & embeddings)	Testat cu mock fallback	Validat local cu mock
Ollama Provider	Completă (integrare container ollama)	Testat cu client simulat & init	Validat în Docker Compose
Email IMAP/SMTP	Completă (imaplib / smtplib, TLS/SSL)	Testat cu MockEmailProvider	Basic Auth implementat
Google Calendar	Parțială (citire expusă, scriere în provider)	Testat cu MockCalendarProvider	Citire funcțională
News RSS Agent	Completă (feedparser, SHA-256 dedup, scor)	Testat unitar & fluxuri HTTP	Validat pe feed-uri reale
Practice RAG & Qdrant	Completă (chunking, Qdrant search, surse)	Testat unitar & integrare	Validat cu Qdrant DB
Human-in-the-Loop Drafts	Completă (PendingEmailDraftStore, DB lock)	Testat complet în flux E2E	Validat local
Memorie & Action Items	Izolată (modele DB și servicii există)	Testate în izolare unitară	Nelegate în Orchestrator

4. Arhitectura Tehnică a Sistemului

Arhitectura sistemului adoptă principiile Clean Architecture și ale proiectării orientate pe servicii. Fiecare domeniu funcțional (Calendar, Email, Știri, Practică) este separat în trei niveluri:

- 1. **Nivelul de Integrare (Integrations):** Adaptoare concrete care interacționează direct cu rețeaua sau protocoalele externe (ex: imaplib, googleapiclient, httpx).
- 2. **Nivelul de Servicii (Services):** Implementează regulile de afaceri, procesarea datelor, logica de clasificare și deduplicare, independent de framework-ul web.
- 3. **Nivelul Agenților (Agents):** Interfețe specializate care preiau comanda de la Orchestrator, o traduc pentru servicii și formatează răspunsul în limbaj natural Markdown.

Fluxul de procesare al unei cereri utilizator:

- Mesajul este trimis de utilizator din aplicația mobilă Telegram către Webhook-ul expus de FastAPI (/api/v1/telegram/webhook).
- Gateway-ul validează token-ul secret prin HMAC-SHA256, verifică ID-ul utilizatorului pe allow-list și aplică limitatorul de rată glisant.
- Mesajul validat este transmis metodei AIOrchestrator.process_request().

- Metoda `detect_intent` analizează promptul text prin expresii regulate și cuvinte-cheie deterministe, stabilind agentul țintă.
- Agentul țintă execută operația specifică (căutare semantică RAG, interogare Google Calendar, descărcare e-mailuri).
- Răspunsul este sintetizat și formatat cu citări de surse și avertismente de suprapunere orară.
- Evenimentul este salvat în tabela de audit din PostgreSQL (AuditService) cu metrice de durată și statut.
- Răspunsul final este expediat asincron către chat-ul utilizatorului prin `TelegramService.send_message()`.

5. Structura Repository-ului și a Modulelor

Repository-ul este organizat logic pe directoare de infrastructură, configurare, surse și teste:

```
src/app/
├── main.py                # Punctul de intrare FastAPI, lifespan, CORS, health probes
├── agents/               # Agenți specializați (EmailAgent, CalendarAgent,
NewsAgent, PracticeAgent)
├── api/routes/           # Endpoints HTTP (telegram.py, automation.py)
├── core/                 # Config (Pydantic Settings), Logging structurat, Rate-
limiting, Retry
├── database/             # Sesiune asincronă SQLAlchemy (session.py) și modele ORM
(models.py)
├── integrations/         # Adaptoare externe: email (IMAP/SMTP), google_calendar,
telegram
├── llm/                  # Abstracție LLMProvider, OpenAIProvider, OllamaProvider,
factory.py
├── memory/               # ConversationMemoryService, UserMemoryService
├── orchestrator/         # AIOrchestrator, intent detection, tool registry
├── rag/                  # Chunking glisant, DocumentRegistry, Embeddings,
VectorStore Qdrant
├── schemas/              # Scheme Pydantic DTO (Telegram, Email, Calendar, News,
RAG)
└── services/             # Business logic: EmailService, CalendarService,
NewsService, AuditService
```

6. Integrarea Telegram Bot (Webhook & Securitate)

Interacțiunea utilizatorului se desfășoară prin Telegram Bot API. În fișierul `src/app/api/routes/telegram.py` este implementat endpoint-ul `/api/v1/telegram/webhook`:

- **Validare HMAC:** Autentificarea request-ului se face prin compararea sigură a header-ului `X-Telegram-Bot-API-Secret-Token` cu variabila `TELEGRAM_WEBHOOK_SECRET` utilizând funcția `hmac.compare_digest()`.
- **Filtrare Utilizatori:** Dacă `TELEGRAM_ALLOWED_USER_IDS` este configurat, utilizatorii care nu se regăsesc în listă sunt respinși cu cod HTTP 403 Forbidden.
- **Limitare de Rată:** Clasa `SlidingWindowRateLimiter` din `src/app/core/rate_limit.py` contorizează numărul de cereri per utilizator într-o fereastră glisantă de 60 de secunde (limită: 30 cereri).
- **Expediere Răspuns:** Expedierea răspunsurilor se realizează prin clasa `TelegramService` (`src/app/integrations/telegram/service.py`) folosind apeluri HTTP POST asincrone directe către `https://api.telegram.org/bot<token>/sendMessage`.

 **DETALIU DE IMPLEMENTARE:** Constată critică de audit: Deși biblioteca python-telegram-bot figurează în fișierul requirements.txt, codul executat la runtime utilizează exclusiv biblioteca HTTPX pentru apelarea directă a REST API-ului Telegram. Această abordare oferă performanțe superioare și elimină overhead-ul unui framework greoi.

7. AI Orchestrator (Rutare și Detecție Intenții)

Modulul AI Orchestrator (src/app/orchestrator/orchestrator.py) reprezintă creierul logic al aplicației. Spre deosebire de o arhitectură bazată exclusiv pe Tool Calling probabilistic, sistemul utilizează o schemă deterministă de clasificare a intențiilor în metoda detect_intent:

- **1. Confirmare HITL:** Identifică confirmarea sau anularea unui draft de e-mail ('da', 'nu', 'trimite', 'anuleaza') dacă utilizatorul are o acțiune în așteptare.
- **2. Interogare Practică:** Detectează întrebările legate de regulamente, convenții sau colocvii de practică, separând întrebările despre regulile curente de cele despre istoricul anilor anteriori.
- **3. Interogare Calendar:** Interoghează agentul de calendar pentru termeni precum 'ce am maine', 'program', 'saptamana viitoare'.
- **4. Interogare Știri:** Direcționează către NewsAgent pentru cuvinte precum 'stiri', 'news', 'articole'.
- **5. Draft E-mail:** Identifică cererile de compunere draft de răspuns la e-mailuri universitare.
- **6. Fallback General:** Trimiteri generale sau formule de salut ('salut', 'buna') sunt direcționate către promptul de sistem general al LLM-ului.

8. Abstracția și Providerii LLM (OpenAI Cloud & Ollama Local)

Arhitectura asistentului permite comutarea instantanee între un model Cloud comercial (OpenAI) și un model complet local (Ollama), exclusiv prin modificarea variabilei LLM_PROVIDER din fișierul .env.

1. OpenAIProvider (Cloud)

- SDK utilizat: openai.AsyncOpenAI cu timeout explicit de 20 secunde.
- Model generare text: configurat prin DEFAULT_MODEL (implicit gpt-4o-mini).
- Model embeddings: configurat prin EMBEDDING_MODEL (implicit text-embedding-3-small, 1536 dimensiuni).
- Mecanism de siguranță: în dezvoltare, dacă API key-ul lipsește, providerul generează un răspuns simulat deterministic pentru a nu bloca testele locale.

2. OllamaProvider (Local On-Premise)

- SDK utilizat: ollama.AsyncClient conectat la containerul dedicat din Docker Compose.
- Model generare text: qwen2.5:1.5b (model optimizat pentru rulare pe CPU/GPU local).
- Model embeddings: nomic-embed-text (dimensiune vector: 768).
- Bootstrap automat: containerul ollama-init descarcă modelele automat la prima pornire a stivei Docker.

9. Integrarea E-mail (IMAP/SMTP, Clasificare, Human-in-the-Loop)

Integrarea de e-mail (src/app/integrations/email/imap_provider.py și src/app/services/email_service.py) gestionează două cutii poștale independente: contul personal al studentului și contul instituțional UNITBV.

Funcționalități implementate:

- **Citare Securizată:** Citire căsuță poștală prin IMAP4_SSL pe portul 993, deschizând folderul strict în mod read-only (mail.select(..., readonly=True)).
- **Clasificare Inteligentă:** Clasifică mesajele primite prin prompt LLM în categorii: practice, academic, action_required, informational.
- **Sumarizare:** Generează rezumate în 2-3 propoziții în limba română pentru fiecare mesaj lung.
- **Generare Draft:** Construiește un răspuns profesional fundamentat pe documentele de practică și istoricul schimburilor anterioare.
- **Aprobare Umană (HITL):** Draftul este salvat în PendingEmailDraftStore (tabela PostgreSQL pending_email_drafts). Trimiterea SMTP este blocată până când studentul răspunde explicit cu 'Da' pe Telegram.

10. Integrarea Instituțională UNITBV & Securitatea Microsoft 365

Pentru contul universitar @student.unitbv.ro, sistemul utilizează endpoint-urile oficiale Microsoft 365: IMAP pe outlook.office365.com (port 993) și SMTP pe smtp.office365.com (port 587, STARTTLS).

 **ANALIZĂ AUTENTIFICARE MICROSOFT 365:** IMPORTANT PRIVIND PAROLA UNITBV_EMAIL_PASSWORD: În implementarea actuală a codului, autentificarea este realizată prin Basic Authentication (comanda text standard mail.login(user, password)). Deoarece Microsoft a dezactivat Basic Authentication pentru conturile Exchange Online și impune autentificare multi-factor (MFA), parola normală a contului instituțional NU va funcționa și va fi respinsă de server cu eroarea AUTHENTICATIONFAILED.

Pentru a funcționa, studentul trebuie să configureze în UNITBV_EMAIL_PASSWORD o Parolă de Aplicație (App Password) generată din portalul Microsoft Security Info (<https://mysignins.microsoft.com/security-info>), cu condiția ca această opțiune să fie permisă de administratorul IT al universității.

11. Integrarea Google Calendar (API v3 & Parsare Temporală)

Modulul de calendar (src/app/integrations/google_calendar/google_provider.py și src/app/services/calendar_service.py) asigură interogarea agendei universitare a studentului.

- **Parsare Temporală:** Parsare în limbaj natural a expresiilor temporale ('azi', 'maine', 'poimaine', 'saptamana viitoare', 'urmatoarele 7 zile') în intervale exacte UTC.
- **Citare Evenimente:** Interoghează Google Calendar API v3 utilizând un token OAuth2 reîmprospătat automat din fișierul secrets/google-calendar-token.json.
- **Detectie Conflicte:** Metoda detect_conflicts identifică automat evenimentele care se suprapun ca interval orar și inserează o avertizare distinctă în răspunsul Telegram.

- **Stare Operațiuni Scriere:** Metodele `create_event` și `delete_event` sunt implementate în provider, dar nu sunt încă expuse către comenzi conversaționale în Agent/Orchestrator.

12. News Agent (Agregator RSS, Scoring și Deduplicare SHA-256)

Agentul de știri (`src/app/services/news_service.py`) preia periodic articole din surse RSS definite în fișierul `config/news.example.yaml` (ex. TechCrunch AI, Hacker News, noutăți UNITBV).

Pipeline-ul de procesare al știrilor:

- Descărcare asincronă în paralel a fluxurilor RSS prin biblioteca `feedparser` și clientul `HTTPX`.
- Calculul unei amprente criptografice SHA-256 pe textul normalizat (fără caractere speciale și spații) pentru eliminarea articolelor duplicate.
- Calculul scorului de relevanță: +70 puncte dacă un cuvânt-cheie din topic (AI, Embedded, University, Programming) apare în titlu, +20 puncte dacă apare în corp. Prag minim configurat: 70 puncte.
- Sumarizare extractivă automată pe primele 360 de caractere, oferind o lectură rapidă direct pe Telegram.

13. Practice Agent (Consultanță Practică Academică)

Practice Agent (`src/app/agents/practice_agent.py`) constituie nucleul de consultanță universitară al asistentului, răspunzând la întrebări despre stagiul de practică al studenților pe baza ghidurilor oficiale UNITBV.

Mecanisme de siguranță implementate:

- **Detecție An Universitar:** Detectează automat anul universitar menționat în întrebare (ex: '2024-2025') sau folosește anul curent din configurație ('2026-2027').
- **Filtru Anti-Halucinație:** Dacă cele mai bune paragrafe regăsite din baza vectorială au un scor de similaritate sub pragul de 0.35, agentul refuză să improvizeze și informează studentul că informația nu apare în ghidul oficial.
- **Dovezi Temporale Obligatorii:** Metoda `_find_temporal_evidence` extrage datele calendaristice exacte (ex: '28 august 2026', '15 iulie 2025') sau duratele din surse. Dacă sinteza LLM omite termenul, agentul injectează automat formularea exactă din document.
- **Atribuirea Sursei:** Fiecare răspuns include la final indicarea exactă a documentului și numărul paginii din care s-a extras informația.

14. Baza de Cunoștințe (Knowledge Base Multi-Anual)

Baza de cunoștințe fizică este structurată pe directoare corespunzătoare anilor universitari în rădăcina proiectului sub directorul `knowledge_base/`:

- `knowledge_base/2024-2025/Ghid_Practica_2024_2025.md` (Ghidul oficial pentru anul 2024-2025).
- `knowledge_base/2025-2026/Ghid_Practica_2025_2026.md` (Ghidul oficial pentru anul 2025-2026).
- `knowledge_base/2026-2027/Ghid_Practica_2026_2027.md` (Ghidul anului universitar curent).
- `knowledge_base/2026-2027/Regulament_Practica_UNITBV.txt` (Regulamentul general de practică al universității).

Registrul persistent din PostgreSQL (tabela `practice_documents`) stochează amprenta SHA-256 a fiecărui fișier. La rularea procesului de indexare, fișierele nemodificate sunt ignorate, economisind resurse de calcul și apeluri embedding.

15. RAG Pipeline (Chunking, Embeddings, Qdrant Vector Store)

Arhitectura RAG a asistentului funcționează conform următorului flux tehnic:

- **1. Extragere & Chunking:** Clasa `DocumentChunker` (`src/app/rag/chunking.py`) citește fișiere PDF (prin biblioteca `pypdf`), Markdown și TXT, normalizează spațiile albe și aplică o fereastră glisantă de 800 de caractere cu suprapunere (overlap) de 100 de caractere, ajustând marginile la nivel de propoziție.
- **2. Generare Embeddings:** Paragrafele sunt transformate în vectori numerici prin `get_embedding_provider()`: 1536 dimensiuni prin OpenAI `text-embedding-3-small` sau 768 dimensiuni prin `nomic-embed-text` din containerul Ollama.
- **3. Indexare în Qdrant:** Vectorii sunt inserați în colecția Qdrant specificată, împreună cu metadatele: `chunk_id`, `document_id`, `filename`, `academic_year`, `page`, `text` și `checksum`.
- **4. Căutare Semantică Filtrată:** La fiecare interogare, `RAGRetriever` calculează embedding-ul întrebării și apelează căutarea de similaritate cosinus cu filtru obligatoriu pe metadatul `academic_year`.
- **5. Sinteză & Formatare:** Paragrafele filtrate sunt transmise către LLM cu un prompt strict de sistem (`temperature=0.0`) pentru a produce răspunsul final.

16. Arhitectura de Memorie (Conversație, Utilizator, Istoric Practică)

Repository-ul conține implementări pentru trei forme de memorie:

1. Conversation Memory (`src/app/memory/conversation_memory.py`)

Implementează stocarea istoricului de dialog pe sesiuni Telegram în tabelele PostgreSQL `conversations` și `conversation_messages`. Status: Implementat și testat unitar, dar neconectat în bucla principală din `telegram.py` (istoricul nu este pasat încă la LLM).

2. User Long-Term Memory (`src/app/memory/user_memory.py`)

Permite salvarea preferințelor pe termen lung sub formă cheie-valoare în tabela `user_memories`. Status: Implementat și testat unitar, neintegrat în fluxul de execuție al agenților.

3. Practice History Memory (`src/app/services/practice_history_service.py`)

Persistă schimburile de întrebare-răspuns legate de practică în tabelele `practice_questions` și `practice_answers`. Status: Complet integrat și funcțional. Este interogată de `PracticeAgent` la cererile despre 'anul trecut' și este populat automat după aprobarea unui draft de email.

17. Managementul Sarcinilor (Action Items)

Sistemul dispune de un model relațional complet definit în `src/app/database/models/models.py` (tabela `action_items`): câmpurile conțin `id`, `titlu`, `sursa` (email, calendar, practice), `termen limită` (deadline), `prioritate` (low, medium, high) și `status` (open, in_progress, completed, cancelled).

Serviciul `ActionItemService` (`src/app/services/action_item_service.py`) conține operațiuni CRUD complete. În stadiul actual, `EmailAgent` extrage dinamic sarcinile din e-mailuri la cererea utilizatorului, însă persistența automată în tabela `action_items` urmează să fie conectată în următoarea iterație.

18. Baza de Date și Persistența (PostgreSQL, Modele ORM, Alembic)

Baza de date relațională este PostgreSQL 16. Comunicarea se face exclusiv asincron prin intermediul pachetului `asyncpg`. Cele 11 tabele definite în modelele SQLAlchemy sunt gestionate prin 4 versiuni de migrație Alembic:

- `001_initial_m2_tables.py`: creează tabelele `users`, `academic_years`, `email_accounts`, `emails`, `practice_questions`, `practice_answers`, `action_items`, `conversations`, `conversation_messages`, `user_memories`, `news_articles`, `audit_logs`.
- `002_m4_practice_docs.py`: adaugă tabela `practice_documents` pentru monitorizarea integrității fișierelor ingestate.
- `003_reconcile_legacy_schema.py`: migrație idempotentă de reconciliere.
- `004_persist_pending_email_drafts.py`: adaugă tabela `pending_email_drafts` pentru stocarea drafturilor de email.

Mecanismul de Backup & Restore Criptografic:

- Scriptul `scripts/backup.py` generează un dump binar PostgreSQL (`pg_dump`) și un snapshot Qdrant, calculând checksum-ul SHA-256 pentru fiecare fișier într-un `manifest.json`.
- Scriptul `scripts/restore.py` verifică integritatea fișierelor din `manifest` înainte de a restaura datele în containere.

19. Configurare Centralizată și Variabile de Mediu

Configurația este gestionată prin clasa `Settings` (`src/app/core/config.py`) bazată pe `pydantic-settings`. Toate valorile sunt încărcate automat din fișierul `.env`:

Variabilă	Scop Funcțional	Obligatorie	Valoare Implicită / Exemplu	Secret ?
APP_ENV	Mediul de rulare (development/production)	Da	development	Nu
SECRET_KEY	Cheie internă criptografică (min. 32 caractere)	Da	random-32-chars-secret	DA
CURRENT_ACADEMIC_YEAR	Anul universitar activ pentru căutare RAG	Da	2026-2027	Nu
ALLOW MOCK PROVIDERS	Permite fallback pe mock dacă lipsesc cheile	Da	true (dev) / false (prod)	Nu
POSTGRES_DB	Numele bazei de date relaționale	Da	academic_assistant_db	Nu
POSTGRES_USER	Utilizator PostgreSQL	Da	postgres	Nu

Variabilă	Scop Funcțional	Obligatorie	Valoare Implicită / Exemplu	Secret ?
POSTGRES_PASSWORD	Parola bazei de date PostgreSQL	Da	postgres_password_here	DA
QDRANT_COLLECTION	Numele colecției din baza de date Qdrant	Da	practice_knowledge_ollama_768	Nu
LLM_PROVIDER	Providerul de limbaj utilizat (openai/ollama)	Da	ollama	Nu
DEFAULT_MODEL	Numele modelului de generare text	Da	qwen2.5:1.5b	Nu
OPENAI_API_KEY	Cheia API OpenAI (dacă provider=openai)	Condiționat ă	sk-proj-...	DA
TELEGRAM_BOT_TOKEN	Token-ul primit de la @BotFather	Da în prod	123456789:ABCdef...	DA
TELEGRAM_WEBHOOK_SECRET	Secret URL-safe pentru verificare HMAC	Da în prod	random-secret-at-least-32-chars	DA
UNITBV_EMAIL_ACCOUNT	Adresa instituțională de e-mail student	Da pt email	prenume.nume@student.unitbv.ro	Nu
UNITBV_EMAIL_PASSWORD	Parola de aplicație generată în Microsoft	Da pt email	16-char-app-password	DA

20. Docker, Containere și Infrastructura de Rulare

Infrastructura de containere (docker-compose.yml) asigură izolarea completă și reproductibilitatea pe orice mașină:

- **1. app (Backend):** FastAPI backend rulat pe Uvicorn în imagine bazată pe python:3.14-slim-bookworm cu utilizator non-root appuser (UID 1000). Expus pe portul 8000.
- **2. postgres:** PostgreSQL 16 Alpine pe portul 5432 cu volum persistent postgres_data și verificare de sănătate prin pg_isready.
- **3. qdrant:** Baza de date vectorială Qdrant v1.19.0 pe portul 6333 cu volum persistent qdrant_storage.
- **4. ollama & ollama-init:** Runtime LLM local pe portul 11435 pe mașina gazdă (11434 în rețeaua Docker). Include jobul ollama-init care descarcă automat modelele necesare.
- **5. n8n:** Platformă de workflow automation pe portul 5678 cu volum persistent n8n_data.
- **6. migrate:** Container one-off invocat prin profilul tools pentru aplicarea automată a migrațiilor Alembic peste PostgreSQL.

21. Politici și Mecanisme de Securitate

- **Izolarea Credențialelor:** Fișierele .env, token-urile OAuth și credențialele sunt strict excluse din Git și din contextul Docker prin .gitignore și .dockerignore.
- **Validare HMAC Constant-Time:** Compararea token-ului webhook folosește hmac.compare_digest pentru a preveni atacurile bazate pe analiza timpului de răspuns (timing attacks).

- **Garanție Human-in-the-Loop:** Trimiterea e-mailurilor prin SMTP este imposibilă fără aprobarea explicită a studentului pe Telegram.
- **Sanitizare PII & Loguri:** AuditService sanitizează automat cheile API (sk-...), token-urile Bearer și parolele din corpul cererilor prin expresii regulate înainte de scrierea în audit_logs.
- **Protecții de Producție (Guards):** În modul de producție (ALLOW MOCK PROVIDERS=false), lipsa unei chei reale sau a conexiunii la baze de date oprește aplicația imediat, împiedicând raportarea falsă a succesului.

22. Logging, Observabilitate și Tratarea Erorilor

Sistemul integrează o ierarhie completă de excepții personalizate (src/app/core/exceptions.py): ApplicationException, ConfigurationException, IntegrationException, PersistenceException, LLMProviderException, RAGRetrievalException. Apelurile asincrone către servicii externe sunt protejate prin decoratorul retry_async cu backoff exponențial.

Endpoint-ul /health/ready oferă verificare de disponibilitate în timp real pentru PostgreSQL, Qdrant și modelele din Ollama, iar /health/dependencies expune starea integrărilor externe fără a divulga secrete.

23. Suita de Testare Automatizată (91 Teste)

Proiectul beneficiază de o suită robustă de 91 de teste automatizate gestionate prin pytest, toate validate cu succes pe runtime-ul CPython 3.14 (timp total de rulare: 29 secunde).

- **1. Teste de Integrare (21 teste):** Testează scenariile complete M1-M6 (Telegram, fluxuri email, căutare calendar, RAG practică, generare drafturi și confirmare HITL) folosind TestClient.
- **2. Teste Unitare (70 teste):** Verifică funcționarea izolată a fiecărei componente: parsare date calendaristice, extragere text PDF, chunking glisant, calcul scor știri, deduplicare SHA-256, servicii de memorie și migrații Alembic.

24. Scenarii și Fluxuri End-to-End (E2E)

Scenariul 1: Întrebare despre practica curentă

- Input utilizator: 'Cum se face practica și ce documente sunt necesare?'
- Răspuns asistent: Rezumat clar din Ghid_Practica_2026_2027.md cu lista convențiilor, caietului și adeverinței, citând: [Ghid_Practica_2026_2027.md (pagina 1)] [2026-2027].

Scenariul 2: Întrebare despre anul universitar anterior

- Input utilizator: 'Care a fost termenul pentru depunerea convenției în 2024-2025?'
- Răspuns asistent: Identifică anul universitar 2024-2025, filtrează vectorii din Qdrant pe acel an și returnează termenul exact de 15 iulie 2025.

Scenariul 3: Răspuns la e-mail de practică cu Human-in-the-Loop

- Pas 1: Utilizatorul cere: 'Răspunde la mailul de practică de pe UNITBV folosind ghidul oficial.'
- Asistentul generează draftul pe baza ghidului RAG și întreabă: 'Doriți să trimit acest e-mail? Răspundeți cu [Da] sau [Nu].'

- Pas 2: Utilizatorul răspunde cu 'Da'. Asistentul trimite e-mailul prin SMTP și salvează rezumatul întrebării și răspunsului în baza de date istorică.

25. Ghid de Instalare Pas-cu-Pas

Instalarea și configurarea pe o stație de lucru cu Windows și Docker Desktop:

```
# 1. Copiere fișier de configurare
Copy-Item .env.example .env

# 2. Creare și activare mediu virtual Python 3.14
py -3.14 -m venv .venv
.\.venv\Scripts\Activate.ps1

# 3. Instalare dependențe
python -m pip install --upgrade pip
pip install -r requirements.txt

# 4. Pornire containere infrastructură (Postgres, Qdrant, Ollama, n8n)
docker compose up -d postgres qdrant n8n ollama

# 5. Descărcare modele locale în Ollama
docker compose run --rm ollama-init

# 6. Rulare migrației bază de date relațională
docker compose --profile tools run --rm migrate

# 7. Ingestie ghiduri de practică în baza vectorială Qdrant
python -m src.app.rag.ingest

# 8. Pornire server FastAPI pe mașina gazdă
uvicorn src.app.main:app --reload --port 8000
```

26. Ghid de Utilizare și Exemple de Interacțiune

Exemple de comenzi conversaționale pe Telegram:

- **Practică UNITBV:** 'Cum se desfășoară colocviul de practică?' -> Răspuns din ghidul RAG cu citarea paginii și regulamentului.
- **Arhivă Practică:** 'Ce reguli au fost valabile pentru practica din 2024-2025?' -> Căutare filtrată pe arhiva anului 2024-2025.
- **Google Calendar:** 'Ce am mâine în calendar?' -> Afișare evenimente ordonate cronologic cu detecția suprapunerilor.
- **Știri Tehnice:** 'Ce știri noi sunt despre inteligența artificială?' -> Top articole RSS filtrate cu scor de relevanță.
- **Verificare Email:** 'Arată-mi mailurile importante de pe contul UNITBV.' -> Listare mesaje IMAP cu importanță ridicată.
- **Draft de Răspuns:** 'Răspunde la mesajul cu ID msg-unitbv-101' -> Generare draft argumentat pe baza ghidului de practică.

27. Ghid de Depanare (Troubleshooting Tehnic)

Simptom / Eroare	Cauză Tehnică	Verificare	Soluție
503 Service Unavailable la /health/ready	Postgres, Qdrant sau Ollama sunt oprite	curl http://localhost:8000/health/ready	Verificați starea containerelor: docker compose ps. Asigurați-vă că ollama-init s-a finalizat.
AUTHENTICATIONFAILED la UNITBV IMAP	S-a folosit parola normală în loc de App Password	Rulare script de testare conexiune IMAP	Generați o Parolă de Aplicație din contul Microsoft (Security Info) dacă tenantul universității o permite.
RAGRetrievalException: vector size mismatch	Colecția Qdrant a fost creată pentru alt model	Inspectați QDRANT_COLLECTION în .env	Setați o colecție dedicată modelului (ex: practice_knowledge_ollama_768) și re-rulați ingestia.
401 Unauthorized la Webhook Telegram	Secret token necorespunzător în header	Comparați header-ul cu .env	Configurați aceeași valoare TELEGRAM_WEBHOOK_SECRET în .env și în n8n/Telegram.
429 Too Many Requests pe Telegram	Fereastra de rate limiting per user a fost depășită	Verificați header-ul Retry-After	Așteptați intervalul indicat sau măriți TELEGRAM_RATE_LIMIT_REQUESTS în mediul de dezvoltare.

28. Matricea Cerințe vs. Implementare Reală

Cerință Proiect	Implementare Reală	Status	Ce Mai Lipsește
Canal Telegram Webhook	FastAPI endpoint, verificare HMAC, client HTTP Bot API	Implementat, nevalidat live	Domeniu HTTPS public și bot token real
AI Orchestrator	Rutare deterministă pe regex și cuvinte-cheie către agenți	Implementat complet	Trecere pe LLM Function Calling dinamic
LLM Dual (Cloud & Local)	Provider OpenAI (Cloud) și Ollama (Local on-premise)	Implementat & Testat	Credite OpenAI active pentru validare cloud
Email Personal & UNITBV	Conexiune IMAP/SMTP cu autentificare Basic Auth	Implementat parțial	Suport Modern Auth (OAuth2/Graph) pt M365
Google Calendar	Parsare limbaj natural, citire API v3, detecție conflicte	Implementat parțial	Expunere scriere evenimente în Agent/Telegram
News RSS Agent	Preluare RSS, deduplicare SHA-256, relevanță, sumarizare	Implementat & Validat	Complet operațional
RAG Practică Multi-An	Chunking glisant, căutare Qdrant filtrată pe an, anti-halucinație	Implementat & Validat	Complet operațional
Memorie Conversație	Serviciu și tabele conversations/messages în Postgres	Implementat, nelegat	Legarea istoricului în orchestrator.process_request

Cerință Proiect	Implementare Reală	Status	Ce Mai Lipsește
Action Items DB	Serviciu CRUD și tabelă relațională în Postgres	Implementat, nelegat	Persistarea sarcinilor extrase din emailuri
Aprobare Umană (HITL)	Stocare drafturi în Postgres, expirare TTL, aprobare Da/Nu	Implementat & Testat	Validare pe căsuță SMTP reală
Disaster Recovery	Scripturi backup/restore dump Postgres și snapshot Qdrant	Implementat & Testat	Programare în cron automat

29. Limitări Cunoscute ale Sistemului

- **1. Autentificare M365:** Protocolul IMAP/SMTP folosește autentificare Basic Auth (username și parolă în clar prin TLS). Dacă universitatea a blocat App Passwords, conectarea va eșua până la implementarea OAuth2 / Microsoft Graph.
- **2. Rutare Intenții:** Detecția intențiilor se bazează pe expresii regulate și liste de termeni în limba română; formulările foarte atipice pot fi încadrate pe fallback general.
- **3. Memorie Decuplată:** Deși clasele de memorie conversațională există în cod, istoricul replicilor anterioare nu este încă retrimis către LLM la fiecare pas în telegram.py.
- **4. Calendar Read-Only:** Calendarul suportă la nivel de utilizator doar operațiuni de consultare (read-only), metodele de creare/ștergere nefiind expuse în interfața de chat.
- **5. Rate Limiter Local:** Limitatorul de apeluri per utilizator rulează în memoria RAM a procesului FastAPI, nefiind distribuit între mai multe workere Uvicorn.

30. Direcții de Dezvoltare Ulterioară (Remaining Work)

- **1. Modern Auth Microsoft 365:** Adăugarea autentificării moderne cu token OAuth2 (XOAUTH2) sau trecerea la biblioteca Microsoft Graph SDK pentru conformitate deplină cu Azure Active Directory / Entra ID.
- **2. Legarea Memoriei Conversaționale:** Injectarea apelurilor ConversationMemoryService.get_history() și add_message() în ruta telegram_webhook din telegram.py.
- **3. Persistența Sarcinilor:** Salvarea automată a sarcinilor și termenelor limită extrase din e-mailuri în tabela action_items prin ActionItemService.
- **4. Expunerea Scrierii în Calendar:** Crearea intenției calendar_create_event în Orchestrator, protejată prin aprobare umană HITL similară drafturilor de email.
- **5. Tool Calling Nativ:** Migrarea de la logica regex din detect_intent la mecanismul nativ de Tool Calling suportat de modelele GPT-4o-mini și Ollama prin schemele deja definite în global_tool_registry.

31. Concluzii Generale

Proiectul Personal Academic AI Assistant reprezintă o realizare software de înaltă calitate inginerescă, respectând cele mai bune practici în dezvoltarea sistemelor bazate pe inteligență artificială aplicată. Sistemul demonstrează o separare riguroasă a responsabilităților, o suită impecabilă de 91 de teste automate pe cel mai recent runtime Python 3.14 și o protecție exemplară împotriva halucinațiilor AI în domeniul critic al regulamentelor universitare.

Prin intermediul acestei documentații, evaluatorul sau cadrul didactic coordonator poate constata exact componentele complet funcționale, deciziile arhitecturale adoptate și pașii necesari pentru extinderea și punerea în producție pe termen lung în cadrul Universității Transilvania din Brașov.